

1 前言

树是一种非线性的数据结构，用它能很好地描述有分支和层次特性的数据集合。树型结构在现实世界中广泛存在，如社会组织机构的组织关系图就可以用树型结构来表示。树在计算机领域中也有广泛应用，如在编译系统中，用树表示源程序的语法结构。在数据库系统中，树型结构是数据库层次模型的基础，也是各种索引和目录的主要组织形式。在许多算法中，常用树型结构描述问题的求解过程、所有解的状态和求解的对策等。在这些年的国内、国际信息学奥赛、大学生程序设计比赛等竞赛中，树型结构成为参赛者必备的知识之一，尤其是建立在树型结构基础之上的搜索算法。

在树型结构中，二叉树是最常用的结构，它的分支个数确定、又可以为空、并有良好的递归特性，特别适宜于程序设计，因此也常常将一般树转换成二叉树进行处理。

2 树

2.1 定义

树 (Tree) 是由 n ($n \geq 0$) 个结点组成的有限集合，以及结点之间关系组成的集合构成的结构。 $n=0$ 时称为空树。在任意一颗非空树中：

- 1) 有且仅有一个特定的称为根 (Root) 的结点；
- 2) 当 $n > 1$ 时，其余结点可分为若干 m ($m > 0$) 个互不相交的有限集 T_1 、 T_2 、……、 T_m ，其中每一个集合本身又是一棵树，并且称为根的子树。

此外，树的定义还需要强调以下两点：

- 1) $n > 0$ 时根结点是唯一的，不可能存在多个根结点，数据结构中的树只能有一个根结点。对于一棵树的根结点来说，其没有前驱结点。
- 2) 除了根结点外，每个结点都有且仅有一个直接前驱结点。包括根结点在内，每个结点都可以有任意后继结点。
- 3) $m > 0$ 时，子树的个数没有限制，但它们一定是互不相交的。

示例树：

图 2.1 为一棵普通的树：(树形表示法——借助自然界中一棵倒置的树的形状来表示数据元素之间层次关系的方法)

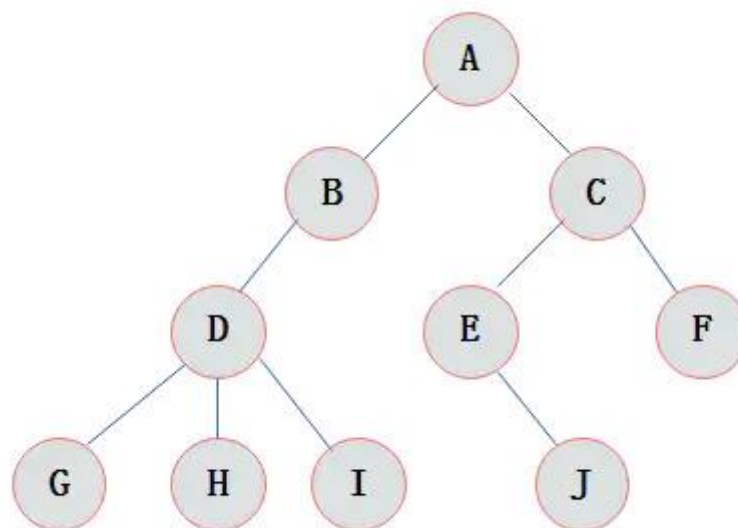


图 2.1 普通树

由树的定义可以看出，树的定义使用了递归的方式。递归在树的学习过程中起着重要作用，如果对于递归还不是十分了解，应当加强递归算法的练习。

2.2 结点的度

结点拥有的子树数目称为结点的**度**。

图 2.2 中标注了图 2.1 所示树的各个结点的度。

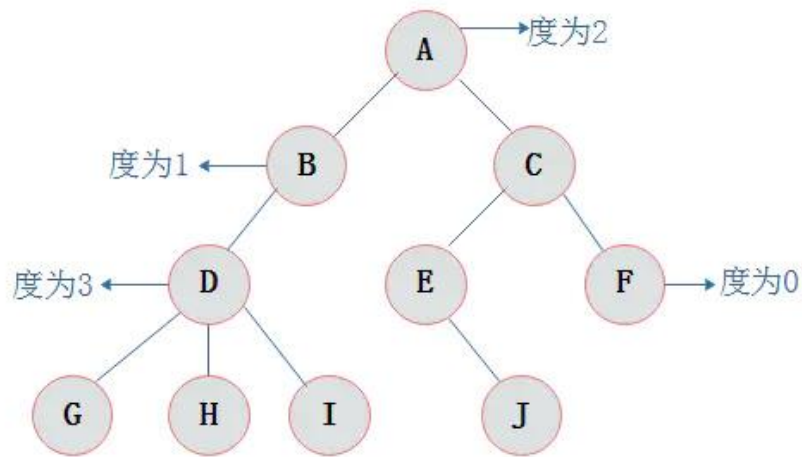


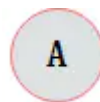
图 2.2 度示意图

2.3 结点概念

结点是数据结构中的基础，是构成复杂数据结构的基本组成单位。

2.3.1 树结点声明

本系列文章中提及的结点专指树的结点。例如：结点 A 在图中表示为：



2.3.2 叶结点、分支结点

叶结点为位于整棵树的底部、度为 0 的结点。**分支结点**即度非 0 的结点。

2.3.3 父结点、子结点、祖先结点、后代

父结点 i 为结点 j 的前驱，子结点 j 为结点 i 的后继结点。祖先结点为所有延伸出去的前驱结点，后代为所有延伸出去的后继结点。

2.4 结点关系

结点子树的根结点为该结点的**孩子结点**。相应该结点称为孩子结点的**双亲结点**。

图 2.2 中，A 为 B 的双亲结点，B 为 A 的孩子结点。

同一个双亲结点的孩子结点之间互称**兄弟结点**。
图 2.2 中，结点 B 与结点 C 互为兄弟结点。

2.4 结点层次

从根开始定义起，根为第一层，根的孩子为第二层，以此类推。
图 2.3 表示了图 2.1 所示树的层次关系

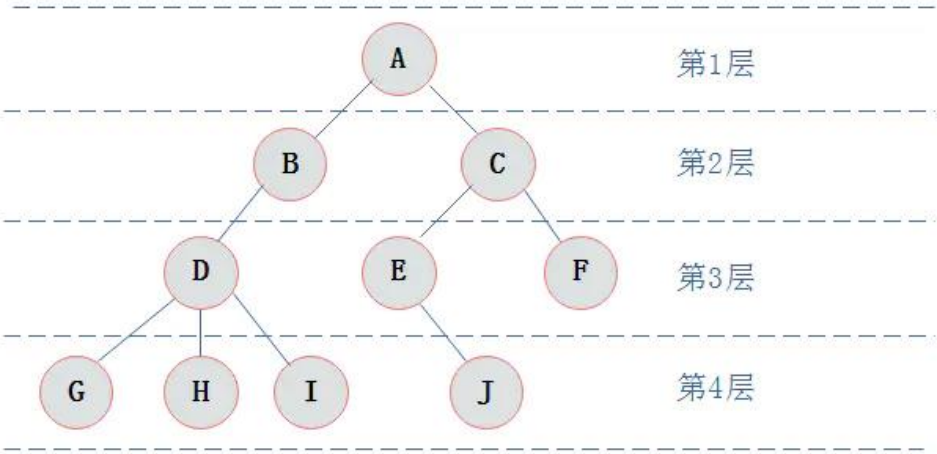


图 2.3 层示意图

2.5 树的深度

树中结点的最大层次数称为树的深度或高度。图 2.1 所示树的深度为 4。

2.6 树林(森林)

$m \geq 0$ 棵不相交的树组成的树的集合。

3 二叉树

3.1 定义

二叉树是 $n(n \geq 0)$ 个结点的有限集合，该集合或者为空集（称为空二叉树），或者由一个根结点和两棵互不相交的、分别称为根结点的左子树和右子树组成。图 3.1 展示了一棵普通二叉树：

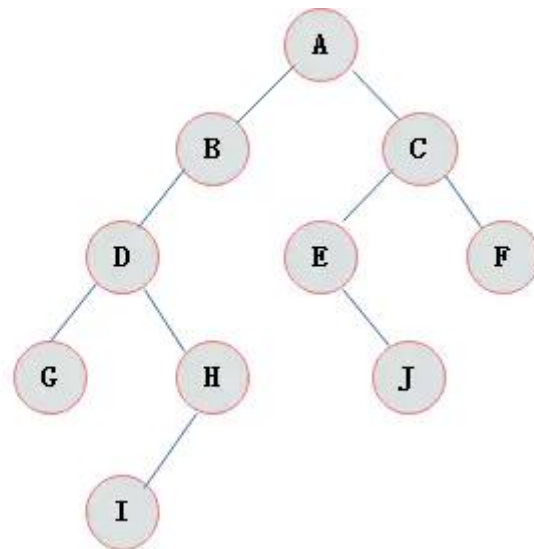


图 3.1 二叉树

3.2 二叉树特点

由二叉树定义以及图示分析得出二叉树有以下特点：

- 1) 每个结点最多有两颗子树，所以二叉树中不存在度大于 2 的结点。
- 2) 左子树和右子树是有顺序的，次序不能任意颠倒。
- 3) 即使树中某结点只有一棵子树，也要区分它是左子树还是右子树。

3.3 二叉树性质

- 1) 在二叉树的第 i 层上最多有 2^{i-1} 个节点。（ $i \geq 1$ ）
- 2) 二叉树中如果深度为 k , 那么最多有 $2^k - 1$ 个节点。（ $k \geq 1$ ）
- 3) $n_0 = n_2 + 1$ n_0 表示度数为 0 的叶结点数， n_2 表示度数为 2 的节点数。
- 4) 在完全二叉树中，具有 n 个节点的完全二叉树的深度为 $\lceil \log_2 n \rceil + 1$, 其中 $\lceil \log_2 n \rceil$ 是向下取整。
- 5) 若对含 n 个结点的完全二叉树从上到下且从左至右进行 1 至 n 的编号，则对完全二叉树中任意一个编号为 i 的结点有如下特性：

- (1) 若 $i=1$, 则该结点是二叉树的根，无双亲，否则，编号为 $\lfloor i/2 \rfloor$ 的结点为其双亲结点；
- (2) 若 $2i > n$, 则该结点无左孩子， 否则，编号为 $2i$ 的结点为其左孩子结

点；

(3) 若 $2i+1 > n$ ，则该结点无右孩子结点， 否则，编号为 $2i+1$ 的结点为其右孩子结点。

3.4 斜树

斜树：所有的结点都只有左子树的二叉树叫左斜树。所有结点都是只有右子树的二叉树叫右斜树。这两者统称为斜树。

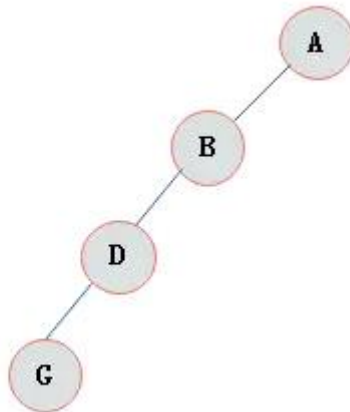


图 3.2 左斜树

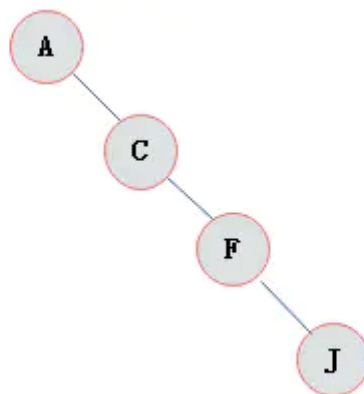


图 3.3 右斜树

3.5 满二叉树

满二叉树：在一棵二叉树中。如果所有分支结点都存在左子树和右子树，并且所有叶子都在同一层上，这样的二叉树称为满二叉树。

满二叉树的特点有：

- 1) 叶子只能出现在最下一层。出现在其它层就不可能达成平衡。
- 2) 非叶子结点的度一定是 2。
- 3) 在同样深度的二叉树中，满二叉树的结点个数最多，叶子数最多。

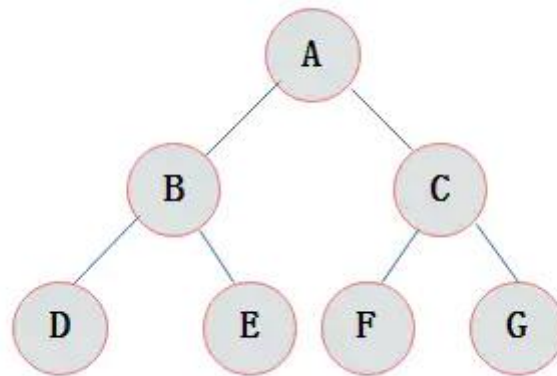


图 3.4 满二叉树

3.6 完全二叉树

完全二叉树：对一颗具有 n 个结点的二叉树按层编号，如果编号为 i ($1 \leq i \leq n$) 的结点与同样深度的满二叉树中编号为 i 的结点在二叉树中位置完全相同，则这棵二叉树称为完全二叉树。

图 3.5 展示一棵完全二叉树

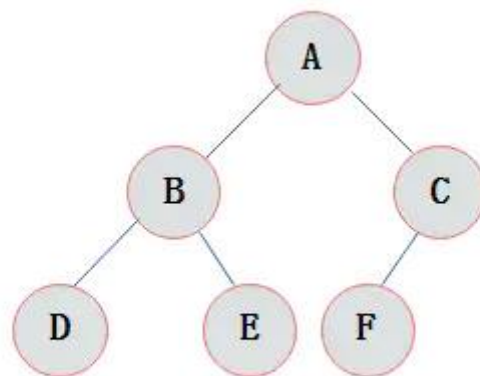


图 3.5 完全二叉树

特点：

- 1) 叶子结点只能出现在最下层和次下层。
- 2) 最下层的叶子结点集中在树的左部。
- 3) 倒数第二层若存在叶子结点，一定在右部连续位置。
- 4) 如果结点度为 1，则该结点只有左孩子，即没有右子树。
- 5) 同样结点数目的二叉树，完全二叉树深度最小。

注：满二叉树一定是完全二叉树，但反过来不一定成立。

3.7 二叉树的存储结构

3.7.1 顺序存储

二叉树的顺序存储结构就是使用一维数组存储二叉树中的结点，并且结点的存储位置，就是数组的下标索引。

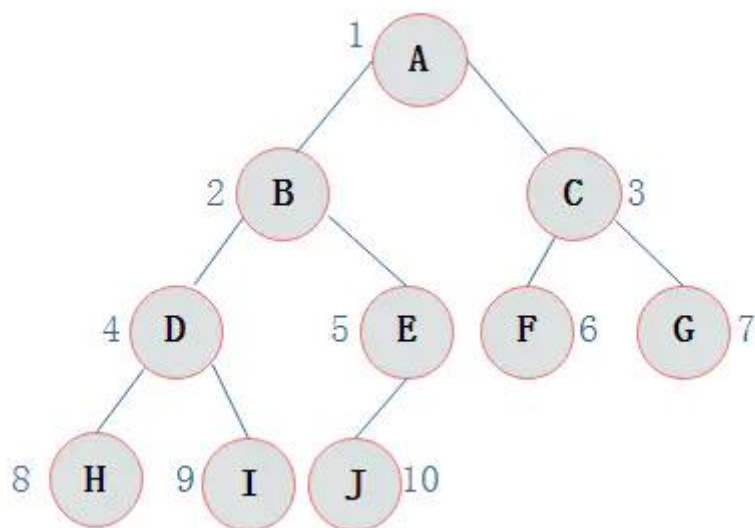


图 3.6

图 3.6 所示的一棵完全二叉树采用顺序存储方式，如图 3.7 表示：



图 3.7 顺序存储

由图 3.7 可以看出，当二叉树为完全二叉树时，结点数刚好填满数组。
那么当二叉树不为完全二叉树时，采用顺序存储形式如何呢？例如：对于图 3.8 描述的二叉树：

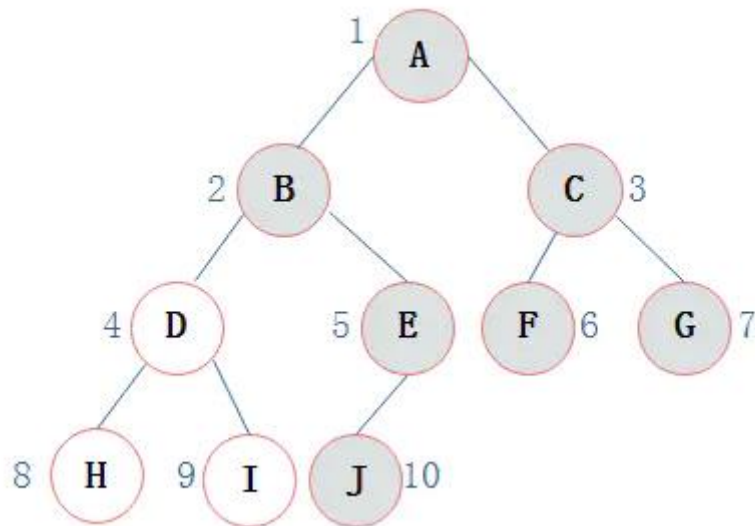


图 3.8.png

其中浅色结点表示结点不存在。那么图 3.8 所示的二叉树的顺序存储结构如图 3.9 所示：

1	2	3	4	5	6	7	8	9	10
A	B	C	∧	E	F	G	∧	∧	J

图 3.9

其中，∧表示数组中此位置没有存储结点。此时可以发现，顺序存储结构中已经出现了空间浪费的情况。
那么对于图 3.3 所示的右斜树极端情况对应的顺序存储结构如图 3.10 所示：

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	∧	C	∧	∧	∧	F	∧	∧	∧	∧	∧	∧	∧	J

图 3.10

由图 3.10 可以看出，对于这种右斜树极端情况，采用顺序存储的方式是十分浪费空间的。因此，顺序存储一般适用于完全二叉树。

3.7.2 二叉链表

既然顺序存储不能满足二叉树的存储需求，那么考虑采用链式存储。由二叉树定义可知，二叉树的每个结点最多有两个孩子。因此，可以将结点数据结构定义为一个数据和两个指针域。表示方式如图 3.11 所示：



图 3.11

定义结点代码：

```
typedef struct BiTNode{
    TElemType data; //数据
    struct BiTNode *lchild, *rchild; //左右孩子指针
} BiTNode, *BiTree;
```

则图 3.6 所示的二叉树可以采用图 3.12 表示。

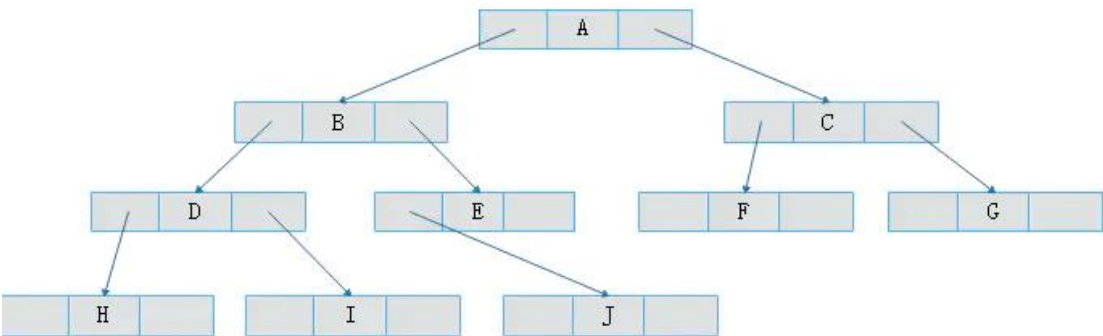


图 3.12

图 3.12 中采用一种链表结构存储二叉树，这种链表称为二叉链表。

3.8 二叉树遍历

二叉树的遍历一个重点考查的知识点。

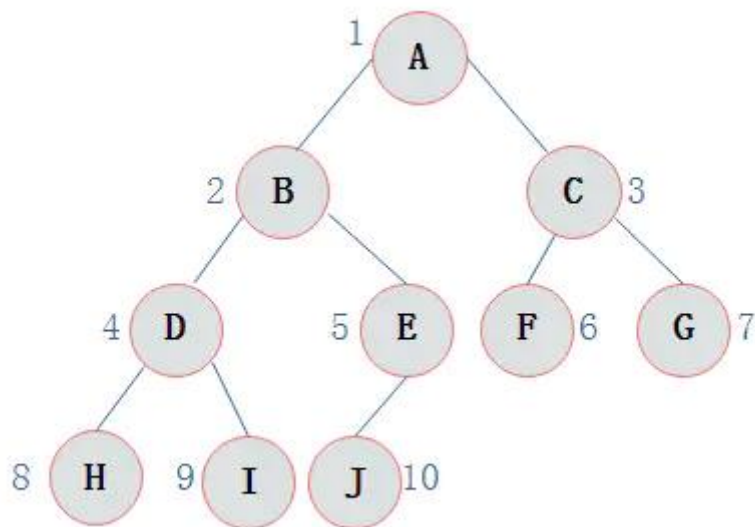
3.8.1 定义

二叉树的遍历是指从二叉树的根结点出发，按照某种次序依次访问二叉树中的所有结点，使得每个结点被访问一次，且仅被访问一次。
二叉树的访问次序可以分为四种：

前序遍历
中序遍历
后序遍历
层序遍历

3.8.2 前序遍历

前序遍历通俗的说就是从二叉树的根结点出发,当第一次到达结点时就输出结点数据,按照先向左在向右的方向访问。



3.13

图 3.13 所示二叉树访问如下:

从根结点出发,则第一次到达结点 A,故输出 A;

继续向左访问,第一次访问结点 B,故输出 B;

按照同样规则,输出 D,输出 H;

当到达叶子结点 H,返回到 D,此时已经是第二次到达 D,故不在输出 D,进而向 D 右子树访问, D 右子树不为空,则访问至 I,第一次到达 I,则输出 I;

I 为叶子结点,则返回到 D, D 左右子树已经访问完毕,则返回到 B,进而到 B 右子树,第一次到达 E,故输出 E;

向 E 左子树,故输出 J;

按照同样的访问规则,继续输出 C、F、G;

则 3.13 所示二叉树的前序遍历输出为:

ABDHIEJCFG

3.8.3 中序遍历

中序遍历就是从二叉树的根结点出发，当第二次到达结点时就输出结点数据，按照先向左在向右的方向访问。

图 3.13 所示二叉树中序访问如下：

从根结点出发，则第一次到达结点 A，不输出 A，继续向左访问，第一次访问结点 B，不输出 B；继续到达 D，H；
到达 H，H 左子树为空，则返回到 H，此时第二次访问 H，故输出 H；
H 右子树为空，则返回至 D，此时第二次到达 D，故输出 D；
由 D 返回至 B，第二次到达 B，故输出 B；
按照同样规则继续访问，输出 J、E、A、F、C、G；

则 3.13 所示二叉树的中序遍历输出为：

HDIBJEAFCG

3.8.4 后序遍历

后序遍历就是从二叉树的根结点出发，当第三次到达结点时就输出结点数据，按照先向左在向右的方向访问。

图 3.13 所示二叉树后序访问如下：

从根结点出发，则第一次到达结点 A，不输出 A，继续向左访问，第一次访问结点 B，不输出 B；继续到达 D，H；
到达 H，H 左子树为空，则返回到 H，此时第二次访问 H，不输出 H；
H 右子树为空，则返回至 H，此时第三次到达 H，故输出 H；
由 H 返回至 D，第二次到达 D，不输出 D；
继续访问至 I，I 左右子树均为空，故第三次访问 I 时，输出 I；
返回至 D，此时第三次到达 D，故输出 D；
按照同样规则继续访问，输出 J、E、B、F、G、C，A；

则图 3.13 所示二叉树的后序遍历输出为：

HIDJEBFGCA

虽然二叉树的遍历过程看似繁琐，但是由于二叉树是一种递归定义的结构，故采用递归方式遍历二叉树的代码十分简单。

递归实现代码如下：

```
/*二叉树的前序遍历递归算法*/  
void PreOrderTraverse(BiTree T)  
{  
    if(T==NULL)  
        return;  
    printf("%c", T->data); /*显示结点数据，可以更改为其他对结点操作*/  
}
```

```

    PreOrderTraverse(T->lchild);    /*再先序遍历左子树*/
    PreOrderTraverse(T->rchild);    /*最后先序遍历右子树*/
}

/*二叉树的中序遍历递归算法*/
void InOrderTraverse(BiTree T)
{
    if(T==NULL)
        return;
    InOrderTraverse(T->lchild); /*中序遍历左子树*/
    printf("%c", T->data); /*显示结点数据，可以更改为其他对结点操作*/
    InOrderTraverse(T->rchild); /*最后中序遍历右子树*/
}

/*二叉树的后序遍历递归算法*/
void PostOrderTraverse(BiTree T)
{
    if(T==NULL)
        return;
    PostOrderTraverse(T->lchild); /*先后序遍历左子树*/
    PostOrderTraverse(T->rchild); /*再后续遍历右子树*/
    printf("%c", T->data); /*显示结点数据，可以更改为其他对结点操作*/
}

```

3.8.5 层次遍历

层次遍历就是按照树的层次自上而下的遍历二叉树。针对图 3.13 所示二叉树的层次遍历结果为：

ABCDEFGHIJ

层次遍历的详细方法可以参考二叉树的按层遍历法。

3.8.6 遍历常考考点

对于二叉树的遍历有一类典型题型。

1) 已知前序遍历序列和中序遍历序列，确定一棵二叉树。

例题：若一棵二叉树的前序遍历为 ABCDEF，中序遍历为 CBAEDF，请画出这棵二叉树。

分析：前序遍历第一个输出结点为根结点，故 A 为根结点。早中序遍历中根结点处于左右子树结点中间，故结点 A 的左子树中结点有 CB，右子树中结点有 EDF。如图 3.14 所示：

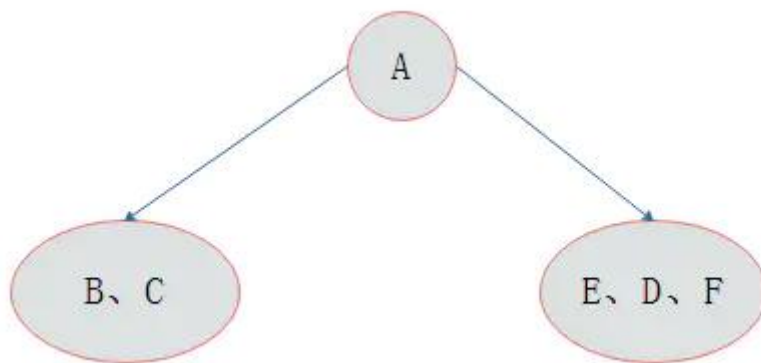


图 3.14

按照同样的分析方法，对 A 的左右子树进行划分，最后得出二叉树的形态如图 3.15 所示：

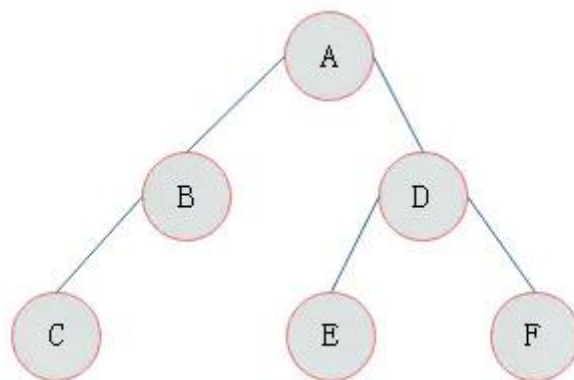


图 3.15.png

2) 已知后序遍历序列和中序遍历序列，确定一棵二叉树。

后序遍历中最后访问的为根结点，因此可以按照上述同样的方法，找到根结点后分成两棵子树，进而继续找到子树的根结点，一步步确定二叉树的形态。

注：已知前序遍历序列和后序遍历序列，不可以唯一确定一棵二叉树。

4 结语

通过上述的介绍，已经对于二叉树有了初步的认识。本篇文章介绍的基础知识希望读者能够牢牢掌握，并且能够在脑海中建立一棵二叉树的模型，为后续学习打好基础。